Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-214БВ-24

Студент: ГеннихА.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата:17.09.25

Москва, 2025

Постановка задачи

Вариант 7.

Разработать программу на языке Си, состоящую из двух процессов (родительского и дочернего), которые взаимодействуют через разделяемую память (shared memory) с синхронизацией через семафоры.

Общий метод и алгоритм решения

Использованные системные вызовы:

1. write - запись данных в файловый дескриптор (в stdout, stderr и в pipe)
2. read - чтение данных из файлового дескриптора (из stdin и из файла)
3. exit - немедленное завершение процесса с указанным кодом возврата
4. open - открытие файла и получение файлового дескриптора
5. close - закрытие файлового дескриптора
6. pipe - создание канала (pipe) для межпроцессного взаимодействия
7. fork - создание нового процесса (дочернего) путем клонирования текущего
8. dup2 - дублирование файлового дескриптора в указанный дескриптор (перенаправление stdin)
9. execv - замена текущего образа процесса новой программой (запуск child)
10. wait - ожидание изменения состояния дочернего процесса
11. sigaction - установка обработчика сигналов
12. waitpid - ожидание конкретного дочернего процесса с дополнительными опциями
13. fcntl - управление файловым дескриптором (установка неблокирующего режима)
14. access - проверка прав доступа к файлу
15. parse_float - парсинг строки в число с плавающей точкой:
    Преобразует строку в float с помощью strtof
    Проверяет корректность преобразования
    Проверяет, что после числа нет лишних символов
    Возвращает true/false в зависимости от успеха
16. parse_float - парсинг строки в число с плавающей точкой:
    Преобразует строку в float с помощью strtof
    Проверяет корректность преобразования
    Проверяет, что после числа нет лишних символов
    Возвращает true/false в зависимости от успеха

Программа реализует взаимодействие двух процессов через канал (pipe) для обработки числовых данных из файла. Родительский процесс запрашивает имя файла, открывает его и через канал передает содержимое дочернему процессу. Дочерний процесс получает данные через перенаправленный стандартный ввод, построчно извлекает числа и вычисляет их сумму, выводя результаты.

Родитель создает канал (pipe), порождает дочерний процесс (fork), который перенаправляет stdin на канал (dup2) и замещается программой child (execv). Родитель читает файл и через канал передает данные дочернему процессу. Дочерний процесс принимает данные, разбивает на строки, парсит числа и для каждой строки выводит сумму. По завершении родитель ожидает дочерний процесс (wait), закрываются все файловые дескрипторы и обрабатываются возможные ошибки.

Код программы

Parent.c

```
#include <stdint.h>
#include <stdbool.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <string.h>

char CHILD_PROGRAM_NAME[] = "./child";

int main()
{
    char filename[256];
    {
        const char prompt[] = "Enter filename: ";
        write(STDOUT_FILENO, prompt, sizeof(prompt) - 1);

        ssize_t bytes = read(STDIN_FILENO, filename, sizeof(filename) - 1);
        if (bytes <= 0)
        {
            const char msg[] = "error: failed to read filename\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
        }

        if (filename[bytes - 1] == '\n')
        {
            filename[bytes - 1] = '\0';
        }
        else
        {
            filename[bytes] = '\0';
        }
    }

    int32_t file = open(filename, O_RDONLY);
```

```
if (file == -1)
{
    const char msg[] = "error: failed to open file\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

int parent_to_child[2];
if (pipe(parent_to_child) == -1)
{
    const char msg[] = "error: failed to create pipe\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    close(file);
    exit(EXIT_FAILURE);
}

const pid_t child_pid = fork();

switch (child_pid)
{
case -1:
    {
        const char msg[] = "error: failed to spawn new process\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        close(file);
        close(parent_to_child[0]);
        close(parent_to_child[1]);
        exit(EXIT_FAILURE);
    }
    break;

case 0:
    {
        close(parent_to_child[1]);
        close(file);
        if (dup2(parent_to_child[0], STDIN_FILENO) == -1)
        {
            const char msg[] = "error: failed to redirect stdin\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
```

```
        }
        close(parent_to_child[0]);

        char* const args[] = {"child", NULL};
        execv(CHILD_PROGRAM_NAME, args);

        const char msg[] = "error: failed to exec into child program\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
    break;

default:
    {
        close(parent_to_child[0]);

        char buf[4096];
        ssize_t bytes;

        while ((bytes = read(file, buf, sizeof(buf))) > 0)
        {
            int32_t written = write(parent_to_child[1], buf, bytes);

            if (written != bytes)
            {
                const char msg[] = "error: failed to write to pipe\n";
                write(STDERR_FILENO, msg, sizeof(msg) - 1);
                close(file);
                close(parent_to_child[1]);
                wait(NULL);
                exit(EXIT_FAILURE);
            }
        }

        if (bytes < 0)
        {
            const char msg[] = "error: failed to read from file\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
        }
```

```
            close(file);
            close(parent_to_child[1]);

            int status;
            wait(&status);

            if (WIFEXITED(status))
            {
                exit(WEXITSTATUS(status));
            }
            else
            {
                const char msg[] = "error: child terminated abnormally\n";
                write(STDERR_FILENO, msg, sizeof(msg) - 1);
                exit(EXIT_FAILURE);
            }
        }
        break;
    }

    return 0;
}
```

Child.c

```
#include <stdint.h>
#include <stdbool.h>
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <math.h>
#include <float.h>

#define MAX_NUMBERS 100
#define MAX_NUM_LENGTH 64
#define BUFFER_SIZE 4096

bool parse_float(const char* str, float* result)
```

```
{
    char* endptr;
    float value = strtof(str, &endptr);


    if (endptr == str)
    {
        return false;
    }

    while (*endptr != '\0')
    {
        if (!isspace(*endptr))
        {
            return false;
        }
        endptr++;
    }

    if (value == HUGE_VALF || value == -HUGE_VALF || value == 0.0f)
    {
        return true;
    }

    *result = value;
    return true;
}

int32_t float_to_string(float num, char* str, int precision)
{
    if (isnan(num))
    {
        return snprintf(str, MAX_NUM_LENGTH, "nan");
    }

    if (isinf(num))
    {
        if (num > 0)
            return snprintf(str, MAX_NUM_LENGTH, "inf");
        else
```

```
        return snprintf(str, MAX_NUM_LENGTH, "-inf");
    }


    return snprintf(str, MAX_NUM_LENGTH, "%.*f", precision, num);
}

void process_line(const char* line, int32_t length)
{
    float numbers[MAX_NUMBERS];
    int32_t count = 0;

    char num_buffer[MAX_NUM_LENGTH];
    int32_t num_index = 0;

    for (int32_t i = 0; i < length; ++i)
    {
        char c = line[i];

        if ((c >= '0' && c <= '9') ||
            c == '-' || c == '+' ||
            c == '.')
        {
            if (num_index < MAX_NUM_LENGTH - 1)
            {
                num_buffer[num_index++] = c;
            }
        }
        else if (isspace(c))
        {
            if (num_index > 0)
            {
                num_buffer[num_index] = '\0';
                float num;
                if (parse_float(num_buffer, &num) && count < MAX_NUMBERS)
                {
                    numbers[count++] = num;
                }
                num_index = 0;
            }
```

```
        }
    }

    if (num_index > 0)
    {
        num_buffer[num_index] = '\0';
        float num;
        if (parse_float(num_buffer, &num) && count < MAX_NUMBERS)
        {
            numbers[count++] = num;
        }
    }

    float sum = 0.0f;
    for (int32_t i = 0; i < count; ++i)
    {
        sum += numbers[i];
    }

    char result[64];
    int32_t len = float_to_string(sum, result, 6);

    char output[128];
    int32_t output_len = snprintf(output, sizeof(output),
                                  "Sum of %d numbers: %s\n", count, result);

    write(STDOUT_FILENO, output, output_len);
}

int main(void)
{
    char buffer[BUFFER_SIZE];
    char line[BUFFER_SIZE];
    int32_t line_length = 0;
    ssize_t bytes;

    int32_t line_number = 0;

    while ((bytes = read(STDIN_FILENO, buffer, sizeof(buffer))) > 0)
    {
```

```
        for (int32_t i = 0; i < bytes; ++i)
        {
            if (buffer[i] == '\n')
            {
                if (line_length > 0)
                {
                    line_number++;
                    char line_header[32];
                    int32_t header_len = snprintf(line_header, sizeof(line_header),
                                                  "Line %d: ", line_number);
                    write(STDOUT_FILENO, line_header, header_len);

                    process_line(line, line_length);
                    line_length = 0;
                }
            }
            else
            {
                if (line_length < BUFFER_SIZE - 1)
                {
                    line[line_length++] = buffer[i];
                }
            }
        }
    }

    if (bytes < 0)
    {
        const char msg[] = "error: failed to read from stdin\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }



    return 0;
}
```

Протокол работы программы

Тестирование:

Data.txt:

10.5 20.3 30.7
-5.2 15.8 25.1 -35.6
100.0 200.5 300.75
1.1 2.2 3.3 4.4 5.5

```
hoodrich@DESKTOP-DUQ2OCS:~/OSI_LAB/laba_1$ ./parent
Enter filename: data_float.txt
Line 1: Sum of 3 numbers: 61.500000
Line 2: Sum of 4 numbers: 0.100002
Line 3: Sum of 3 numbers: 601.250000
Line 4: Sum of 5 numbers: 16.500000
```

Strace:

```
hoodrich@DESKTOP-DUQ2OCS:~/OSI_LAB/laba_1$ strace -f -tt -o trace_full.txt ./parent
15910 09:12:22.723970 execve("./parent", ["./parent"], 0x7ffed5c5acb0 /* 36 vars */) = 0
15910 09:12:22.725621 brk(NULL)         = 0x55f61756b000
15910 09:12:22.725918 mmap(NULL, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f2a8f120000
15910 09:12:22.726214 access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
15910 09:12:22.726546 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
15910 09:12:22.726776 fstat(3, {st_mode=S_IFREG|0644, st_size=20203, ...}) = 0
15910 09:12:22.726973 mmap(NULL, 20203, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f2a8f11b000
15910 09:12:22.727206 close(3)          = 0
15910 09:12:22.727424 openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6",
O_RDONLY|O_CLOEXEC) = 3
15910 09:12:22.727763 read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832)
= 832
15910 09:12:22.728038 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784,
64) = 784
15910 09:12:22.728269 fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
15910 09:12:22.728486 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784,
64) = 784
15910 09:12:22.728730 mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3,
0) = 0x7f2a8ef09000
15910 09:12:22.728973 mmap(0x7f2a8ef31000, 1605632, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7f2a8ef31000
15910 09:12:22.729230 mmap(0x7f2a8f0b9000, 323584, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) = 0x7f2a8f0b9000
15910 09:12:22.729439 mmap(0x7f2a8f108000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x7f2a8f108000
15910 09:12:22.729687 mmap(0x7f2a8f10e000, 52624, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f2a8f10e000
15910 09:12:22.729988 close(3)          = 0
15910 09:12:22.730219 mmap(NULL, 12288, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f2a8ef06000
15910 09:12:22.730432 arch_prctl(ARCH_SET_FS, 0x7f2a8ef06740) = 0
15910 09:12:22.730663 set_tid_address(0x7f2a8ef06a10) = 15910
15910 09:12:22.730874 set_robust_list(0x7f2a8ef06a20, 24) = 0
15910 09:12:22.731052 rseq(0x7f2a8ef07060, 0x20, 0, 0x53053053) = 0
15910 09:12:22.731413 mprotect(0x7f2a8f108000, 16384, PROT_READ) = 0
15910 09:12:22.731661 mprotect(0x55f5d76da000, 4096, PROT_READ) = 0
15910 09:12:22.731853 mprotect(0x7f2a8f158000, 8192, PROT_READ) = 0
15910 09:12:22.732070 prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
rlim_max=RLIM64_INFINITY}) = 0
```

```
15910 09:12:22.732316 munmap(0x7f2a8f11b000, 20203) = 0
15910 09:12:22.732554 write(1, "Enter filename: ", 16) = 16
15910 09:12:22.732830 read(0, "data_float.txt\n", 255) = 15
15910 09:12:28.862480 openat(AT_FDCWD, "data_float.txt", O_RDONLY) = 3
15910 09:12:28.862689 pipe2([4, 5], 0)  = 0
15910 09:12:28.862881 clone(child_stack=NULL,
flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x7f2a8ef06a10) =
15947
15947 09:12:28.863218 set_robust_list(0x7f2a8ef06a20, 24 <unfinished ...>
15910 09:12:28.863248 close(4 <unfinished ...>
15947 09:12:28.863257 <... set_robust_list resumed>) = 0
15910 09:12:28.863267 <... close resumed>) = 0
15910 09:12:28.863352 read(3,  <unfinished ...>
15947 09:12:28.863411 close(5 <unfinished ...>
15910 09:12:28.863448 <... read resumed>"10.5 20.3 30.7\n-5.2 15.8 25.1 -3"..., 4096) = 75
15947 09:12:28.863511 <... close resumed>) = 0
15910 09:12:28.863547 write(5, "10.5 20.3 30.7\n-5.2 15.8 25.1 -3"..., 75 <unfinished ...>
15947 09:12:28.863587 close(3 <unfinished ...>
15910 09:12:28.863621 <... write resumed>) = 75
15947 09:12:28.863656 <... close resumed>) = 0
15910 09:12:28.863691 read(3,  <unfinished ...>
15947 09:12:28.863727 dup2(4, 0 <unfinished ...>
15910 09:12:28.863762 <... read resumed>"", 4096) = 0
15947 09:12:28.863817 <... dup2 resumed>) = 0
15910 09:12:28.863853 close(3 <unfinished ...>
15947 09:12:28.863888 close(4 <unfinished ...>
15910 09:12:28.863922 <... close resumed>) = 0
15947 09:12:28.863968 <... close resumed>) = 0
15910 09:12:28.864004 close(5 <unfinished ...>
15947 09:12:28.864039 execve("./child", ["child"], 0x7ffeecec1fb8 /* 36 vars */ <unfinished ...>
15910 09:12:28.864127 <... close resumed>) = 0
15910 09:12:28.864218 wait4(-1,  <unfinished ...>
15947 09:12:28.864360 <... execve resumed>) = 0
15947 09:12:28.864494 brk(NULL)         = 0x55f667533000
15947 09:12:28.864752 mmap(NULL, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fd046324000
15947 09:12:28.864959 access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
15947 09:12:28.865148 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
15947 09:12:28.865327 fstat(3, {st_mode=S_IFREG|0644, st_size=20203, ...}) = 0
15947 09:12:28.865519 mmap(NULL, 20203, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fd04631f000
15947 09:12:28.865690 close(3)          = 0
15947 09:12:28.865862 openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6",
O_RDONLY|O_CLOEXEC) = 3
15947 09:12:28.866038 read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832)
= 832
15947 09:12:28.866211 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784,
64) = 784
15947 09:12:28.866382 fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
15947 09:12:28.866554 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784,
64) = 784
15947 09:12:28.866726 mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3,
0) = 0x7fd04610d000
15947 09:12:28.866897 mmap(0x7fd046135000, 1605632, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7fd046135000
15947 09:12:28.867096 mmap(0x7fd0462bd000, 323584, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) = 0x7fd0462bd000
15947 09:12:28.867250 mmap(0x7fd04630c000, 24576, PROT_READ|PROT_WRITE,
```

```
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x7fd04630c000
15947 09:12:28.867429 mmap(0x7fd046312000, 52624, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7fd046312000
15947 09:12:28.867614 close(3)          = 0
15947 09:12:28.867789 mmap(NULL, 12288, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fd04610a000
15947 09:12:28.867985 arch_prctl(ARCH_SET_FS, 0x7fd04610a740) = 0
15947 09:12:28.868172 set_tid_address(0x7fd04610aa10) = 15947
15947 09:12:28.868339 set_robust_list(0x7fd04610aa20, 24) = 0
15947 09:12:28.868509 rseq(0x7fd04610b060, 0x20, 0, 0x53053053) = 0
15947 09:12:28.868756 mprotect(0x7fd04630c000, 16384, PROT_READ) = 0
15947 09:12:28.868988 mprotect(0x55f639d63000, 4096, PROT_READ) = 0
15947 09:12:28.869159 mprotect(0x7fd04635c000, 8192, PROT_READ) = 0
15947 09:12:28.869358 prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
rlim_max=RLIM64_INFINITY}) = 0
15947 09:12:28.869566 munmap(0x7fd04631f000, 20203) = 0
15947 09:12:28.869767 read(0, "10.5 20.3 30.7\n-5.2 15.8 25.1 -3"..., 4096) = 75
15947 09:12:28.869956 write(1, "Line 1: ", 8) = 8
15947 09:12:28.870215 write(1, "Sum of 3 numbers: 61.500000\n", 28) = 28
15947 09:12:28.870436 write(1, "Line 2: ", 8) = 8
15947 09:12:28.870633 write(1, "Sum of 4 numbers: 0.100002\n", 27) = 27
15947 09:12:28.870925 write(1, "Line 3: ", 8) = 8
15947 09:12:28.871161 write(1, "Sum of 3 numbers: 601.250000\n", 29) = 29
15947 09:12:28.871493 write(1, "Line 4: ", 8) = 8
15947 09:12:28.871782 write(1, "Sum of 5 numbers: 16.500000\n", 28) = 28
15947 09:12:28.872061 read(0, "", 4096) = 0
15947 09:12:28.872285 exit_group(0)     = ?
15947 09:12:28.872560 +++ exited with 0 +++
15910 09:12:28.872605 <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) =
15947
15910 09:12:28.872697 --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=15947,
si_uid=1000, si_status=0, si_utime=0, si_stime=0} ---
15910 09:12:28.872830 exit_group(0)     = ?
15910 09:12:28.873124 +++ exited with 0 +++
```

Вывод

В ходе работы приобретены навыки управления процессами и организации межпроцессного взаимодействия через каналы в ОС. Разработанная программа демонстрирует создание процессов (fork), передачу данных через pipe, перенаправление ввода-вывода (dup2), запуск дочерней программы (execv) и синхронизацию (wait). Все системные вызовы обрабатывают ошибки. Дочерний процесс суммирует числа из файла, переданного родителем.